

Configuración de Servicios: DNS

1. Introducción Teórica: La Base de Datos Distribuida

El protocolo DNS (Domain Name System) es básicamente la base de datos distribuida más grande del mundo y el núcleo lógico de cualquier red corporativa o de Internet. Su trabajo es simple: traducir nombres que los humanos entendemos (como `www.carreralinux.com.ar`) a direcciones IP (IPv4 o IPv6) que los routers pueden enrutar.

El árbol de DNS no está centralizado, se estructura en niveles estrictos:

1. **Zona Raíz (Root Zone):** Representada lógicamente por un punto (.) al final de cualquier dominio (ej: `google.com.`). Existen 13 servidores raíz principales distribuidos por todo el mundo (nombrados de la A a la M), replicados mediante enrutamiento Anycast.
 2. **TLD (Top-Level Domains - Dominios de Nivel Superior):** Administrados por registros internacionales (como la IANA). Se dividen en:
 - ~ gTLD (Generics): `.com`, `.org`, `.net`, `.edu`.
 - ~ ccTLD (Country Code): `.ar` (Argentina), `.cl` (Chile), `.es` (España).
 3. **SLD (Second-Level Domains - Dominios de Nivel Secundario):** El nombre comercial o institucional registrado por una entidad (ej: `carreralinux` o `utn`).
 4. **Hosts / Subdominios:** El identificador final del dispositivo o servicio (ej: `www` para el servidor web, `mail` para el correo, `ssh` para administración remota).
-

2. Tipos de Resolución y Flujo de Consultas

Un servidor DNS puede operar bajo distintas mecánicas de resolución, y configurarlas mal en producción es un error crítico de seguridad:

Resolución Directa vs. Reversa

- Resolución Directa: El cliente le entrega un nombre de host al servidor (ej: `servidor.aula1.lan`) y espera recibir su dirección IP (ej: `192.168.122.20`).
- Resolución Reversa: El cliente (o un sistema de auditoría/firewall) entrega una dirección IP y exige saber qué nombre de host está vinculado a ella. Utiliza un dominio especial de la zona raíz llamado `in-addr.arpa`.

Consultas Iterativas vs. Recursivas

- Consulta Recursiva (El cliente al Resolver local): El cliente le dice al servidor DNS: "Necesito la IP de este dominio, no me des una pista, búscala vos y entregame el resultado final". El servidor asume toda la carga de buscar en internet.
- Consulta Iterativa (El servidor a los servidores del mundo): El servidor DNS va saltando de un nodo a otro. Le pregunta a un servidor Raíz, este le dice: "No lo sé, pero tomá la IP del TLD .ar". Luego le pregunta al TLD, y este le dice: "Tomá la IP del servidor autorizado de carreralinux". El proceso se repite hasta dar con la respuesta definitiva.

3. Registros DNS (Resource Records)

Dentro de las tablas de bases de datos de un servidor DNS (llamadas **Archivos de Zona**), la información se estructura mediante registros estandarizados:

Tipo de Registro	Nombre Completo	Función Crítica en Producción
SOA	Start of Authority	El encabezado de la zona. Define los parámetros de sincronización, el correo del administrador, el número de serie de la DB y los tiempos de expiración para los servidores esclavos.
NS	Name Server	Delega la autoridad de la zona. Indica qué servidores de la red son los encargados oficiales de responder por ese dominio.
A	Address (IPv4)	El registro mapeador base. Vincula un nombre de host con una dirección IPv4 (32 bits).
AAAA	Address (IPv6)	Idéntico al registro A, pero vincula el host a una dirección IPv6 nativa (128 bits).

CNAME	Canonical Name	Un alias o seudónimo lógico. Permite que múltiples nombres (ej: ftp.empresa.com y web.empresa.com) apunten al mismo registro maestro A.
MX	Mail Exchanger	Redirección de correo electrónico. Define a qué servidores deben enviarse los emails destinados a ese dominio, ordenados por prioridad numérica.
PTR	Pointer	El registro de la Zona Reversa. Mapea una IP hacia un nombre de host (esencial para que los servidores de correo no marquen tus emails como Spam).

4. Diseños de infraestructura en producción

Configurar un servidor DNS requiere diseñar estrategias para optimizar la velocidad de la red y ocultar la infraestructura interna.

DNS de Horizonte Dividido (Split-Horizon DNS)

Una de las mejores prácticas de seguridad es implementar la arquitectura de Horizonte Dividido, haciendo que un único servidor responda cosas totalmente distintas según quién le pregunte.}}

- **La Vista Interna (LAN):** Un empleado consulta por db.empresa.lan y el DNS local le responde con la IP privada: 192.168.122.45.
- **La Vista Externa (Internet):** Si un atacante externo consulta por ese mismo host, el servidor le responde con NXDOMAIN (No existe) o lo redirige a la IP pública de un Firewall. Esto evita que se filtre la topología interna.

Reenviadores (Forwarders) vs. Pistas Raíz (Root Hints)

Cuando tu servidor no tiene guardada la IP de google.com, tiene dos formas de resolverlo:

- **Root Hints (Por defecto):** Inicia el lento viaje iterativo preguntando a los servidores Raíz, TLD, etc. Genera tráfico excesivo.
- **Forwarders (Lo recomendado):** Le configuras una directiva: "Si no conoces la IP, no busques en el mundo; pregúntale directamente a Cloudflare (1.1.1.1) o Google (8.8.8.8)". Podés usar reenviadores filtrados como Cisco Umbrella para bloquear páginas de malware a nivel de resolución, actuando como un firewall DNS.

El Impacto Operacional del TTL (Time To Live)

El TTL indica a los clientes del mundo durante cuánto tiempo (en segundos) pueden guardar un registro en su caché antes de volver a consultar a tu servidor.

- **TTL Alto (ej: 86400s / 24hs):** Reduce el consumo de CPU y red. Ideal para servidores estables. Desventaja: Si un servidor web cae y cambiás la IP de emergencia, Internet tardará 24hs en enterarse.
- **TTL Bajo (ej: 300s / 5min):** Agilidad extrema para migraciones o balanceo de carga en la nube. Dato Forense: Las Botnets usan técnicas de Fast-Flux DNS con TTLs de 5 segundos para evadir bloqueos de los analistas de seguridad.

EDNS0 y el Límite de UDP

DNS corre históricamente sobre UDP, que tiene un límite de paquete de 512 bytes. Si la respuesta (ej. claves de DNSSEC) supera ese tamaño, el servidor corta el paquete y fuerza al cliente a usar TCP (más lento). EDNS0 (RFC 6891) permite a los servidores modernos aceptar paquetes UDP de hasta 4096 bytes.

5. Enfoque en Ciberseguridad: Amenazas Graves y Mitigaciones

El servidor DNS (comúnmente el demonio BIND9 en Linux) es un objetivo prioritario. Conocer estos vectores separa a un SysAdmin de un Ingeniero de Seguridad:

A. AXFR Leak (Fuga de Información por Transferencia de Zona Insegura)

1. **El vector:** El comando AXFR es un mecanismo legítimo para sincronizar la base de datos completa entre un DNS Maestro y un Esclavo. Si no se restringe, un atacante puede ejecutar: `dig axfr @tu-servidor-dns tu-empresa.com`.
2. **El impacto:** El servidor le regalará al atacante el mapa completo de tu red (IPs de desarrollo, bases de datos ocultas, etc.), facilitando la fase de reconocimiento.
3. **Mitigación:** Configurar directivas estrictas de control de acceso (ACLs) usando `allow-transfer { none; };` y autenticar a los esclavos con llaves criptográficas **TSIG**.

B. DNS Cache Poisoning (Envenenamiento de Caché)

1. **El vector:** Un atacante inyecta un registro falso en la memoria caché del servidor recursivo local antes de que llegue la respuesta legítima del exterior.
2. **El impacto:** Cuando los empleados intentan entrar a `homebanking.banco.com`, el DNS envenenado los enviará a una web de Phishing controlada por el atacante.
3. **Mitigación:** Deshabilitar la recursividad para usuarios externos, forzar Source Port Randomization e implementar **DNSSEC** (firmas criptográficas anti-manipulación).

C. DNS Amplification DoS (Ataques de Amplificación DNS)

1. **El vector:** Aprovechando que DNS usa UDP (sin handshake) y soporta **EDNS0**, el atacante envía consultas de 40 bytes falsificando su IP de origen (colocando la IP de su víctima) pidiendo registros pesados.
 2. **El impacto:** El servidor DNS envía la respuesta de 4000 bytes hacia la víctima. Si el atacante lo hace con miles de servidores vulnerables a la vez, genera un ataque DDoS por reflexión masivo (Factor de amplificación de 100x).
 3. **Mitigación:** Deshabilitar Open Resolvers (no responder recursivamente a internet) y activar políticas RRL (Response Rate Limiting).
-

Parte 2: Implementación de DNS en producción

Este trabajo se realizará en un Debian 13 server, donde se ira explicando cada comando paso a paso para lograr implementar DNS.

Paso 1: IP estática

Al igual que el DHCP, un servidor DNS no puede cambiar de dirección IP jamás. Todos los clientes de tu red van a tener esta IP grabada a fuego en sus configuraciones.

Editaremos el archivo de interfaces: `vim /etc/network/interfaces`

```
source /etc/network/interfaces.d/*

# The loopback network interface
auto lo
iface lo inet loopback

# The primary network interface
allow-hotplug enp1s0
iface enp1s0 inet static
    address 192.168.122.20
    netmask 255.255.255.0
    gateway 192.168.122.1
```

Paso 2: Instalación de BIND9 y herramientas base

BIND9 es el software estándar de la industria. No solo instalamos el motor, sino también sus herramientas de diagnóstico.

~ apt update

~ apt install bind9 bind9utils bind9-doc dnsutils -y

```
root@UTN-Lucka:~# apt update && apt install bind9 bind9utils bind9-doc dnsutils -y
Hit:1 http://deb.debian.org/debian trixie InRelease
Hit:2 http://security.debian.org/debian-security trixie-security InRelease
Hit:3 http://deb.debian.org/debian trixie-updates InRelease
All packages are up to date.
Note, selecting 'bind9-utils' instead of 'bind9utils'
Note, selecting 'bind9-dnsutils' instead of 'dnsutils'
The following package was automatically installed and is no longer required:
  linux-image-6.12.86+deb13-amd64
Use 'apt autoremove' to remove it.

Installing:
  bind9  bind9-dnsutils  bind9-doc  bind9-utils
```

¿Para qué hacemos esto? bind9 es el motor del servidor. bind9utils contiene herramientas administrativas vitales (como rndc y chequeadores de sintaxis de zonas). dnsutils nos instala el comando dig, que es nuestra mejor herramienta para hacerle consultas al servidor y verificar si responde bien. Al instalarse, Debian crea automáticamente un usuario de sistema llamado bind sin privilegios.

Importante!!

Nunca se le hace un Hardening a un servicio mientras está corriendo en memoria. Lo apagamos para manipular sus archivos sin corromper nada.

~ systemctl stop bind9

Paso 3: armado de la Jaula Chroot

Históricamente, BIND9 tuvo vulnerabilidades críticas (desbordamientos de búfer) que permitían a un atacante ejecutar código remoto. Si BIND9 corre libre por el sistema, el atacante toma control de todo el disco duro.

El paquete BIND9 en Debian ya viene programado para correr con un usuario sin privilegios llamado bind. Nuestro trabajo de hardening acá se va a centrar en encerrarlo en el directorio /var/lib/named para que, si un atacante lo vulnera, no pueda ver el resto del disco duro.

1. Creación de la estructura del sistema de archivos aislado:

```
~ mkdir -p /var/lib/named/{etc,dev,var/cache/bind,var/run/named}
```

¿Para qué usamos chroot? Un entorno chroot le hace creer al servicio que esta carpeta es la raíz (/) del disco duro. Por ende, tenemos que recrearle adentro las carpetas básicas (etc, dev, var) para que no colapse al iniciar.

2. Clonación de dispositivos de Hardware virtuales:

```
~ mknod /var/lib/named/dev/null c 1 3
```

```
~ mknod /var/lib/named/dev/random c 1 8
```

```
~ mknod /var/lib/named/dev/urandom c 1 9
```

```
~ chmod 666 /var/lib/named/dev/{null,random,urandom}
```

```
root@UTN-Lucka:~# systemctl stop bind9
root@UTN-Lucka:~# mkdir -p /var/lib/named/{etc,dev,var/cache/bind,var/run/named}
root@UTN-Lucka:~# mknod /var/lib/named/dev/null c 1 3
root@UTN-Lucka:~# mknod /var/lib/named/dev/random c 1 8
root@UTN-Lucka:~# mknod /var/lib/named/dev/urandom c 1 9
root@UTN-Lucka:~# chmod 666 /var/lib/named/dev/{null,random,urandom}
```

¿Por qué hacemos esto? El servidor DNS necesita firmar respuestas criptográficamente (DNSSEC), para lo que requiere generar números aleatorios leyendo hardware (random y urandom). También necesita descartar archivos (null). Como el servicio va a estar enjaulado, no podrá ver la carpeta /dev/ real de Debian. Con el comando mknod clonamos las funciones de estos dispositivos adentro de su jaula y les damos permisos de lectura/escritura (666).

3. Migración de la configuración y Enlace Simbólico:

```
~ mv /etc/bind /var/lib/named/etc/  
~ ln -s /var/lib/named/etc/bind /etc/bind  
~ mkdir -p /var/lib/named/usr/share/dns  
~ cp /usr/share/dns/root.hints /var/lib/named/usr/share/dns/
```

Con el primer comando movemos toda la configuración real original que nos dio la instalación hacia el interior de la jaula. Luego, creamos un acceso directo (symlink). Esto es una práctica de comodidad y estandarización: Nos permite seguir tipeando en /etc/bind/named.conf por costumbre muscular, pero el sistema operativo guarda el cambio físicamente adentro de la jaula. Con los últimos dos clonaremos el archivo de servidores raíz hacia el interior de la jaula.

4. Principio de Menor Privilegio (Asignación de propietarios):

```
~ chown -R bind:bind /var/lib/named/var/*  
~ chown -R bind:bind /var/lib/named/etc/bind  
~ mkdir -p /var/lib/named/run/named  
~ chown -R bind:bind /var/lib/named/run
```

```
root@UTN-Lucka:~# mv /etc/bind /var/lib/named/etc/  
mv: '/etc/bind' and '/var/lib/named/etc/bind' are the same file  
root@UTN-Lucka:~# ln -s /var/lib/named/etc/bind /etc/bind  
root@UTN-Lucka:~# chown -R bind:bind /var/lib/named/var/*  
root@UTN-Lucka:~# chown -R bind:bind /var/lib/named/etc/bind
```

```
root@UTN-Lucka:~# mkdir -p /var/lib/named/run/named  
root@UTN-Lucka:~# chown -R bind:bind /var/lib/named/run
```

Con esto le quitamos la propiedad de estas carpetas a root y se las entregamos al usuario sin privilegios bind. Si el servicio BIND9 es vulnerado, el atacante quedará atrapado en el cuerpo de un usuario débil, e impedirá escalar privilegios o modificar cosas fuera de esas carpetas específicas.

Paso 4: Integración con el Kernel (Systemd y AppArmor)

Por más que le hayamos armado la jaula, si encendemos el servicio ahora, el demonio va a arrancar buscando sus cosas en el /etc/ real y AppArmor lo va a bloquear.

1. Modificar los parámetros de arranque del demonio:

En Debian moderno, BIND9 lee sus variables de arranque desde el archivo /etc/default/named.

```
~ vim /etc/default/named
```

En este archivo buscaremos la variable OPTIONS (suele decir OPTIONS="-u bind") y agregamos la flag -t (que significa chroot) apuntando a nuestra jaula. Queda así:

```
#  
# run resolvconf?  
RESOLVCONF=no  
  
# startup options for the server  
OPTIONS="-u bind -t /var/lib/named -4"
```

2. Crear las excepciones en AppArmor:

Por defecto, AppArmor tiene prohibido que BIND9 toque cosas en /var/lib/named. Tenemos que darle permisos.

```
~ vim /etc/apparmor.d/local/usr.sbin.named
```

```
# Permisos para la jaula Chroot  
/var/lib/named/etc/bind/** r,  
/var/lib/named/var/** rw,  
/var/lib/named/dev/null rw,  
/var/lib/named/dev/random rw,  
/var/lib/named/dev/urandom rw,  
/var/lib/named/run/named/** rw,  
/var/lib/named/usr/share/dns/** r,
```

```
/var/lib/named/etc/bind/** r  
/var/lib/named/var/** rw  
/var/lib/named/dev/null rw  
/var/lib/named/dev/random rw  
/var/lib/named/dev/urandom rw
```

3. Recargar la seguridad y Encender:

Aplicamos las reglas de AppArmor, le avisamos a Systemd de los cambios y encendemos.:

~ systemctl reload apparmor

~ systemctl daemon-reload

~ systemctl start bind9

```
root@UTN-Lucka:~# systemctl reload apparmor
root@UTN-Lucka:~# systemctl start bind9
root@UTN-Lucka:~# systemctl status bind9
● named.service - BIND Domain Name Server
   Loaded: loaded (/usr/lib/systemd/system/named.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-07-20 15:41:45 CDT; 5s ago
     Invocation: ce328e246fd545338a63940906524219
       Docs: man:named(8)
    Main PID: 1225 (named)
      Status: "running"
        Tasks: 6 (limit: 4638)
       Memory: 24.5M (peak: 24.9M)
          CPU: 17ms
      CGroup: /system.slice/named.service
              └─1225 /usr/sbin/named -f -u bind -t /var/lib/named
```

Paso 5: Configuración global y seguridad

Ahora que el servicio funciona, configuramos el núcleo del DNS. Acá aplicamos las mitigaciones contra los ataques que vimos en la teoría (AXFR, Amplificación y Control de Recursividad).

Como hicimos el enlace simbólico (ln -s), nos hace mas facil editar todo cómodamente en /etc/bind/.

1. Editar las Opciones Globales:

~ vim /etc/bind/named.conf.options

Quedaría algo así:

```
acl "red-interna" {
    127.0.0.0/8;
    192.168.122.0/24;
};

options {
    directory "/var/cache/bind";

    // 1. HARDENING: ¿Quién puede consultarme? (Evita DNS Amplification)
    allow-query { red-interna; };

    // 2. HARDENING: ¿Quién puede usarme para buscar en internet? (Cierra el Open Resolver)
    recursion yes;
    allow-recursion { red-interna; };

    // 3. HARDENING: Bloqueo de transferencia de zona (Evita AXFR Leak)
    allow-transfer { none; };

    // 4. OPTIMIZACIÓN: Forwarders (Reenviadores)
    // Si no sabemos la respuesta, le preguntamos a Cloudflare y Google
    forwarders {
        1.1.1.1;
        8.8.8.8;
    };
    forward only;

    // 5. Escucha estricta en IPv4
    listen-on-v6 { none; };
    listen-on port 53 { 127.0.0.1; 192.168.122.20; };

    dnssec-validation auto;
};
```

```
acl "red-interna" {
    127.0.0.0/8;
    192.168.122.0/24;
};
```

```
options {
    directory "/var/cache/bind";

    allow-query { red-interna; };

    recursion yes;
    allow-recursion { red-interna; };

    allow-transfer { none; };

    forwarders {
        1.1.1.1;
        8.8.8.8;
    };
    forward only;
    listen-on-v6 { none; };
    listen-on port 53 { 127.0.0.1; 192.168.122.20; };

    dnssec-validation auto;
};
```

Que es cada cosa que colocamos en este archivo?

1. acl "red-interna" Define una Lista de Control de Acceso (Access Control List). En lugar de escribir a mano todas las redes permitidas en cada regla de seguridad, creás una variable lógica. Acá definimos que nuestro perímetro de confianza absoluto es el propio servidor (127.0.0.0/8) y tu subred KVM (192.168.122.0/24).

2. directory "/var/cache/bind" Es el directorio de trabajo donde BIND escupe archivos temporales, estadísticas y la caché de las consultas resueltas. Gracias a la trampa del chroot que armamos en la Fase 3, cuando BIND intenta escribir en /var/cache/bind, el Kernel redirige físicamente la escritura a la jaula de cristal en /var/lib/named/var/cache/bind.

3. allow-query Define quién tiene permiso a nivel de socket para establecer una conexión UDP/TCP al puerto 53 y preguntarle algo al servidor. Al restringirlo a red-interna, si una IP ajena logra llegar al servidor, el paquete se dropea silenciosamente.

4. recursion y allow-recursion (Cierre del Open Resolver) Es la barrera principal contra el ataque de Amplificación DNS.

recursion yes indica que el servidor actuará como un Resolver: si alguien le pregunta por un dominio que no es suyo (ej: youtube.com), el servidor aceptará el trabajo de ir a buscarlo a internet.

allow-recursion { red-interna; } restringe esa capacidad pesada de cómputo exclusivamente a tus usuarios. Si un atacante externo envía una consulta falsificada (spoofing) pidiendo resolver un dominio externo, el servidor le niega el servicio, evitando convertirse en cómplice de un ataque DDoS por reflexión.

5. allow-transfer (Mitigación AXFR) Bloquea la fuga topológica. La transferencia de zona (AXFR) es la operación para que un servidor Esclavo clone la base de datos completa del Maestro. Como acá no armamos un cluster de DNS, se bloquea con none. Sin esto, un atacante ejecuta dig axfr @192.168.122.20 aula1.lan y se descarga tu archivo de zona con todas las IPs de tus servidores internos.

6. forwarders y forward only (Optimización de enrutamiento) Evita el viaje iterativo a los servidores raíz.

forwarders le dice a tu DNS: "Si te piden un dominio de internet, mandale la consulta directamente a los mega-servidores de Cloudflare (1.1.1.1) y Google (8.8.8.8) porque ellos ya lo tienen en su caché y responden en milisegundos".

forward only le prohíbe terminantemente a tu DNS intentar resolver por su cuenta iterativamente. Si los forwarders no responden, la consulta falla. Esto garantiza que tu DNS no genere tráfico basura hacia los servidores raíz.

7. listen-on y listen-on-v6 (Aislamiento de interfaces) Le indica al demonio en qué placas de red físicas debe "atar" su socket de escucha. Obligás al servicio a escuchar estrictamente en tu IP estática .20 y en el localhost de IPv4. Al clavar listen-on-v6 { none; }, apagás completamente la pila IPv6, eliminando los errores "network unreachable" que viste en el status de Systemd.

8. dnssec-validation auto (Mitigación Cache Poisoning) Activa el motor de criptografía del DNS. Cuando los forwarders te envían la IP de un dominio externo, tu servidor primero verifica la firma de seguridad EDNS0 del paquete. Si un atacante interceptó la conexión en el medio e intentó inyectar una IP falsa (Phishing/Cache Poisoning), la firma digital se rompe y tu servidor descarta la respuesta corrupta.

Paso 6: Zonas lógicas

Acá le decimos a BIND9 de qué dominios es dueño.

1. Editar el archivo de zonas locales:

~ vim /etc/bind/named.conf.local

```
//  
// Do any local configuration here  
//  
// ZONA DIRECTA (Nombre a IP)  
zone "aula1.lan" {  
    type master;  
    file "/etc/bind/db.aula1.lan";  
};  
  
// ZONA REVERSA (IP a Nombre) - El segmento de red al revés  
zone "122.168.192.in-addr.arpa" {  
    type master;  
    file "/etc/bind/db.192.168.122";  
};
```

```
// ZONA DIRECTA (Nombre a IP)  
zone "aula1.lan" {  
    type master;  
    file "/etc/bind/db.aula1.lan";  
};
```

```
// ZONA REVERSA (IP a Nombre) - El segmento de red al revés  
zone "122.168.192.in-addr.arpa" {  
    type master;  
    file "/etc/bind/db.192.168.122";  
};
```

Que significa lo que estamos colocando en el archivo?

Zona Directa (zone "aula1.lan")

Este bloque define la resolución de Nombre a IP.

zone "aula1.lan": Declara el espacio de nombres (dominio) para tu red local. Cualquier consulta que termine en .aula1.lan (ej: servidor.aula1.lan) va a ser procesada por las reglas que defines adentro de este bloque.

type master;; Le indica a BIND9 que este servidor es la Autoridad Primaria (Maestro) para este dominio. Significa que él tiene la base de datos original y editable, y no depende de clonar la información de otro servidor DNS de la red (lo que sería un tipo slave).

file "/etc/bind/db.aula1.ian"; Define la ruta absoluta al archivo de texto físico (el archivo de zona) donde vas a escribir los registros A, CNAME y MX.

Detalle de la Jaula Chroot: Aunque escribís /etc/bind/..., acordate que gracias al entorno Chroot y al symlink que armamos, el proceso va a buscar este archivo físicamente en /var/lib/named/etc/bind/db.aula1.ian.

Zona Reversa (zone "122.168.192.in-addr.arpa")

Este bloque define la resolución inversa, es decir, de IP a Nombre. Es obligatorio en redes corporativas para que herramientas de monitoreo, firewalls o servicios de correo puedan validar la identidad de un host a partir de su IP.

"122.168.192.in-addr.arpa": Esta es la nomenclatura estándar e internacional de internet para declarar redes en reversa.

¿Por qué está al revés? Porque el DNS es jerárquico de derecha a izquierda (Raíz -> TLD -> Dominio). Las direcciones IP se leen de izquierda a derecha (192 es la red general, 168 la subred, 122 el segmento). Para meter una IP en la lógica del árbol DNS, tenés que dar vuelta los octetos de tu red (192.168.122.0/24 se convierte en 122.168.192).

in-addr.arpa: Es el dominio especial de la zona raíz global destinado exclusivamente a resolver direcciones IPv4 de forma reversa.

type master;; Al igual que en la directa, tu servidor es el dueño y creador de esta tabla de mapeo inverso.

file "/etc/bind/db.192.168.122"; La ruta al archivo de texto donde vas a meter los registros tipo PTR (Pointer). Físicamente vive en /var/lib/named/etc/bind/db.192.168.122.

Revisión

Con el comando:

```
named-checkconf /etc/bind/named.conf
```

Si el comando no te devuelve nada, la sintaxis del archivo será correcta.
Si devuelve un error te va a decir en que archivo y en que línea está

Paso 7: Creación de las bases de datos (archivos de zona).

Acá vamos a armar los dos archivos que configuramos en el `named.conf.local`. En estos archivos de texto es importante prestar atención a los puntos finales (.). En BIND9, un nombre que no termina en punto hace que el servidor le concatena el nombre del dominio automáticamente, rompiendo la resolución.

1. Archivo de Zona Directa (Resolución Nombre a IP)

Abrimos el archivo físico donde va a estar la base de datos del dominio:

```
~ vim /etc/bind/db.aula1.lan
```

Estructura:

```
$TTL 86400
```

```
@ IN SOA ns1.aula1.lan. admin.aula1.lan. (  
    2026072001 ; Serial  
    3600      ; Refresh  
    1800      ; Retry  
    604800    ; Expire  
    86400 )   ; Minimum TTL
```

```
; Servidores de Nombres (NS)
```

```
@ IN NS ns1.aula1.lan.
```

```
; Registros A (Resolución a IPv4)
```

```
ns1 IN A 192.168.122.20
```

```
fw IN A 192.168.122.1
```

```
db IN A 192.168.122.45
```

```
www IN A 192.168.122.100
```

```
; Registros CNAME (Alias lógicos)
```

```
ftp IN CNAME www
```

```

$TTL 86400
@ IN SOA ns1.aula1.lan. admin.aula1.lan. (
        2026072001 ; Serial
        3600      ; Refresh
        1800      ; Retry
        604800    ; Expire
        86400 )   ; Minimum TTL

; Servidores de Nombres (NS)
@ IN NS ns1.aula1.lan.

; Registros A (Resolución a IPv4)
ns1 IN A 192.168.122.20
fw IN A 192.168.122.1
db IN A 192.168.122.45
www IN A 192.168.122.100

; Registros CNAME (Alias lógicos)
ftp IN CNAME www

```

En mi caso agregue un par de servidores ficticios (fw, db, www) para ver como seria el mapeo completo.

Que significa cada parametro:

\$TTL 86400: El tiempo de vida global en caché para todos los registros de este archivo (24 horas).

@: Es un atajo de sintaxis. Representa el nombre de la zona actual (aula1.lan.).

IN: Indica la clase "Internet". Es un estándar histórico obligatorio.

SOA: Inicia el encabezado de autoridad. El primer parámetro (ns1.aula1.lan.) es el servidor principal. El segundo (admin.aula1.lan.) es el correo del administrador (el primer punto reemplaza a la @ del mail).

Serial: Es el número de versión de la base de datos. El estándar corporativo es usar la fecha al revés más un número de revisión (AAAAMMDDNN). Cada vez que modifiques una IP en el futuro, tenés que sumar 1 a este número para que los servidores esclavos sepan que hubo un cambio y vengan a sincronizar.

Tiempos (Refresh, Retry, Expire): Controlan la frecuencia con la que los servidores esclavos intentan clonar tu base de datos mediante transferencias AXFR.

ns1 IN A: Acá no pusimos punto al final. Como no tiene punto, BIND le agrega el dominio de forma automática, por lo que internamente lo lee como ns1.aula1.lan..

ftp IN CNAME www: Si alguien busca ftp.aula1.lan, el DNS le dice que es un alias de www, devolviendo la IP .100.

2. Archivo de Zona Reversa (Resolución IP a Nombre)

~ vim /etc/bind/db.192.168.122

\$TTL 86400

```
@ IN SOA ns1.aula1.lan. admin.aula1.lan. (  
    2026072001 ; Serial  
    3600      ; Refresh  
    1800      ; Retry  
    604800    ; Expire  
    86400 )   ; Minimum TTL
```

; Servidores de Nombres (NS)

```
@ IN NS ns1.aula1.lan.
```

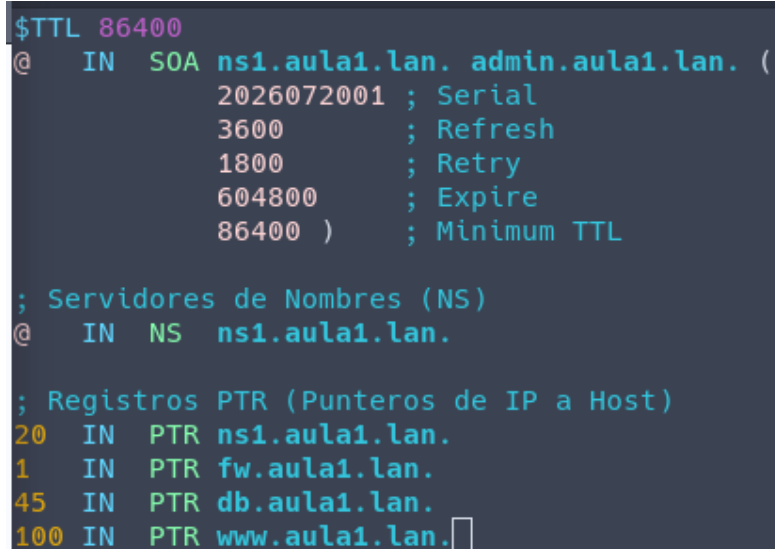
; Registros PTR (Punteros de IP a Host)

```
20 IN PTR ns1.aula1.lan.
```

```
1 IN PTR fw.aula1.lan.
```

```
45 IN PTR db.aula1.lan.
```

```
100 IN PTR www.aula1.lan.
```



```
$TTL 86400  
@ IN SOA ns1.aula1.lan. admin.aula1.lan. (  
    2026072001 ; Serial  
    3600      ; Refresh  
    1800      ; Retry  
    604800    ; Expire  
    86400 )   ; Minimum TTL  
  
; Servidores de Nombres (NS)  
@ IN NS ns1.aula1.lan.  
  
; Registros PTR (Punteros de IP a Host)  
20 IN PTR ns1.aula1.lan.  
1 IN PTR fw.aula1.lan.  
45 IN PTR db.aula1.lan.  
100 IN PTR www.aula1.lan.□
```

Significado de parámetros:

El encabezado SOA y NS es exactamente igual al archivo anterior.

20 IN PTR ns1.aula1.lan.: Solo ponemos el último octeto de la IP (20). Como estamos en la zona 122.168.192.in-addr.arpa, BIND autocompleta el resto. Si una herramienta de auditoría pregunta de quién es la IP .20, el DNS devuelve el nombre completo. Notá que el FQDN al final de la línea tiene el punto obligatorio de cierre.

Paso 8: Verificación de zonas

BIND9 incluye herramientas para validar la integridad de las DB y archivos de configuración antes de recargar el motor.

Para la zona directa:

```
~ named-checkzone aula1.lan /etc/bind/db.aula1.lan
```

Para la zona reversa:

```
~ named-checkzone 122.168.192.in-addr.arpa /etc/bind/db.192.168.122
```

```
zone aula1.lan/IN: loaded serial 2026072001
OK
root@UTN-Lucka:~# named-checkzone 122.168.192.in-addr.arpa /etc/bind/db.192.168.122
zone 122.168.192.in-addr.arpa/IN: loaded serial 2026072001
OK
root@UTN-Lucka:~#
```

```
root@UTN-Lucka:~# systemctl restart bind9
root@UTN-Lucka:~# systemctl status bind9
● named.service - BIND Domain Name Server
   Loaded: loaded (/usr/lib/systemd/system/named.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-07-20 16:42:56 CDT; 12s ago
 Invocation: 3ae1e6edb9cc94ba9864300172c14bf69
    Docs: man:named(8)
   Main PID: 1308 (named)
    Status: "running"
     Tasks: 6 (limit: 4638)
    Memory: 24.1M (peak: 24.7M)
       CPU: 13ms
    CGroup: /system.slice/named.service
            └─1308 /usr/sbin/named -f -u bind -t /var/lib/named -4
```

Si bien el servicio está activo, hay que probar que el motor responde consultas reales como si fuéramos un cliente de la red.

Prueba de zona directa:

```
~ dig @192.168.122.20 www.aula1.lan +short
```

Nos tiene que devolver nuestra IP.

```
root@UTN-Lucka:~# dig @192.168.122.20 www.aula1.lan +short
192.168.122.100
```

Prueba de Zona reversa:

```
~ dig @192.168.122.20 -x 192.168.122.45 +short
```

Nos tiene que devolver el nombre.

```
root@UTN-Lucka:~# dig @192.168.122.20 -x 192.168.122.45 +short
db.aula1.lan.
```